叶环独立集 题解

部分分

Subtask 1：枚举顶点集合

当 $n \le 20$ 时，可以枚举全部 2^n 个顶点子集。对于每个子集，检查树边和叶子环边是否存在两个端点同时被选中的情况，统计合法子集数量即可。

时间复杂度为 $O(2^n n)$。

Subtask 2：枚举叶子状态

设叶子数为 k，且 $k \le 8$。枚举每个叶子是否加入独立集，共有 2^k 种状态。

若叶子环上存在相邻的两个叶子同时被选中，则该状态不合法，直接跳过。否则，固定所有叶子的选取状态，在树上进行普通的独立集 DP。

设

- $f_0[u]$：不选顶点 u 时，以 u 为根的子树的合法方案数；
- $f_1[u]$：选顶点 u 时，以 u 为根的子树的合法方案数。

对于孩子集合 $\mathrm{ch}(u)$，有

$$f_0[u] = \prod_{v \in \mathrm{ch}(u)} \big(f_0[v] + f_1[v] \big),$$

$$f_1[u] = \prod_{v \in \mathrm{ch}(u)} f_0[v].$$

叶子的状态由枚举结果固定，据此调整 DP 初值及转移。每种叶子状态的处理复杂度为 $O(n)$，总时间复杂度为

$$O(2^k n).$$

Subtask 3：星形树

设叶子数为 $m = n - 1$。根被选中时，所有叶子均不能被选中，贡献 1 种方案；根不被选中时，叶子上的选择必须构成环 C_m 的独立集。

设 Fibonacci 数列满足

$$F_0 = 0, \qquad F_1 = 1, \qquad F_i = F_{i-1} + F_{i-2}.$$

环 C_m 的独立集数量为

$$F_{m-1} + F_{m+1}.$$

因此答案为

$$\boxed{1 + F_{m-1} + F_{m+1}}.$$

预处理 Fibonacci 数列即可在线性时间内求解。

满分做法

一、状态设计

注意到叶子按照 DFS 的访问顺序排列，因此任意子树中的叶子在整棵树的叶子序列中必然连续出现。

考虑一棵以 u 为根的子树。为了将不同孩子的子树合并起来，除记录顶点 u 的选取状态外，只需要记录该子树叶子序列的首叶和末叶是否被选中。

定义

$$dp[u][s][a][b]$$

表示满足以下条件的方案数：

- 顶点 u 的选取状态为 s；
- 该子树叶子序列的首叶、末叶的选取状态分别为 a, b；
- 子树内部所有树边和叶子序列中相邻叶子之间的边均满足独立集限制。

其中 $s, a, b \in \{0, 1\}$，状态值均对 $10^9 + 7$ 取模。

因此，每个顶点只有 $2^3 = 8$ 个状态。

二、叶子转移

若 u 是叶子，那么首叶和末叶都是 u，两者的选取状态均等于 s。

因此

$$dp[u][s][a][b] = \begin{cases} 1, a = b = s, \\ 0, \text{otherwise} \end{cases}.$$

三、合并孩子

设顶点 u 的孩子按照编号从小到大依次为

$$v_1, v_2, ..., v_q.$$

固定 u 的选取状态 s，依次合并这些孩子的子树。

初始化：合并第一个孩子。

第一个孩子子树的叶子序列也是当前合并部分的叶子序列，因此枚举其根节点的选取状态 t，得到

$$D[a][b] = \sum_{\substack{t \in \{0,1\} \\ st = 0}} dp[v_1][t][a][b].$$

其中 $st = 0$ 保证父子顶点 u, v_1 不会同时被选中。

转移：合并下一个孩子。

假设当前已合并部分的首叶、末叶选取状态分别为 a, b，方案数为 $D[a][b]$。

设下一个孩子为 v，其子树首叶、末叶的选取状态分别为 c, d，孩子根的选取状态为 t。

合并时需要同时满足两个条件：

1. $st = 0$，即 u 与孩子根 v 不能同时被选中；
2. $bc = 0$，即当前叶子序列的末叶与新加入子树的首叶不能同时被选中。

第二个条件恰好保证新增的子树边界叶子边不会违反独立集限制。

满足上述条件时，合并后的首叶仍为原来的首叶，末叶变为新子树的末叶。因此转移为

$$D'[a][d] \mathrel{+}= D[a][b] \cdot dp[v][t][c][d],$$

其中枚举所有 $b, c, t \in \{0, 1\}$，仅当

$$st = 0, \qquad bc = 0$$

时执行转移。

完成后令 $D \leftarrow D'$，继续处理下一个孩子。

全部孩子合并结束后，得到

$$dp[u][s][a][b] = D[a][b].$$

整个过程中只需枚举常数个二进制状态，因此每条树边的合并复杂度为 $O(1)$。

四、根节点收尾

设整棵树的叶子序列首叶和末叶选取状态分别为 a, b。

如果叶子数 $k = 1$，首叶和末叶实际上是同一个顶点，不需要额外检查叶子环边。

如果 $k \geq 2$，首叶与末叶之间存在一条新增边，因此必须满足

$$ab = 0.$$

特别地，当 $k = 2$ 时，两片叶子之间的相邻边与闭合边重复，但约束仍然是 $ab = 0$。

综上，答案为

$$\boxed{\sum_{s=0}^{1}\sum_{a=0}^{1}\sum_{b=0}^{1} dp[1][s][a][b]\,[k = 1 \vee ab = 0]}$$

其中方括号表示条件指示函数：条件成立时取 1，否则取 0。计算时对 $10^9 + 7$ 取模。

五、复杂度分析

每个顶点有 8 个状态，每条树边只需要进行常数次状态转移。

因此时空复杂度：$O(n)$；

该算法能够满足 $n \le 10^6$ 的数据范围。

实现与边界提示

因为输入保证 $p_i < i$，按编号从 n 到 1 处理时，孩子一定先于父亲完成，无需递归。标准程序把孩子链表建立为编号升序，再按此顺序合并。每个顶点保存 8 个 32 位模数值，三个父子关系数组各保存 $n+1$ 个 32 位整数，主要存储约 44 MB。

只含一片叶子的树必为链, 不能错误地强迫这片叶子不选; 两片叶子的新增边和闭合边虽然重复, 但重复检查同一布尔约束不影响方案数。根自身通常不是叶子, 根选取状态与首末叶状态必须分开记录。模数小于 2^{30}，两项模数相加可安全放入 32 位有符号整数，乘法应使用 64 位。

窗口校准 题解

以下等式均在模 $P = 998244353$ 意义下成立。下标、长度和引用计数仍是普通整数。

窗口条件与唯一性

相邻两个窗口相减，只留下一个进入窗口的元素和一个离开窗口的元素:

$$W_{i+1} - W_i = a_{i+k} - a_i.$$

操作一保留的前 $k+1$ 项确定了前两个窗口 W_l, W_{l+1}，因此公差必须是

$$c = W_{l+1} - W_l = a_{l+k} - a_l.$$

从左到右，对每个尚未确定的元素赋值

$$a_j = a_{j-k} + c \qquad (l+k+1 \le j \le r).$$

右侧元素在赋值时已经确定。每个相邻窗口之差都等于 c，所以该递推得到一个合法结果; 任何合法结果又必须满足同一递推，所以结果唯一。

当 $k = r - l$ 时，区间内只有两个长度为 k 的窗口，且整个区间都被保留，操作恒等。$k = 1$ 时，校准后的原序列本身构成等差数列。

这个递推已经给出了每次线性时间的模拟算法。要处理一般的大规模数据, 需要避免逐项生成长区间。

转为一阶差分的周期延拓

补充 $a_0 = 0$，定义

$$d_i = a_i - a_{i-1} \qquad (1 \le i \le n).$$

两个相邻窗口公差相等，等价于

$$\begin{aligned} 0 &= (a_{i+k+1} - a_{i+1}) - (a_{i+k} - a_i) \\ &= d_{i+k+1} - d_{i+1} \qquad (l \le i \le r-k-1). \end{aligned}$$

换元后，约束恰好是

$$d_j = d_{j-k} \qquad (l+k+1 \le j \le r).$$

由于前 $k+1$ 个原序列元素固定，种子

$$d_{l+1}, d_{l+2}, ..., d_{l+k}$$

也固定。于是只需把差分区间 $[l+1, r]$ 替换成该种子反复出现所得序列的前 $L = r - l$ 项。d_l 保持不变。

替换范围包含原本不变的种子；这有利于统一构造。它与题目允许改变的原数组后缀 $[l+k+1, r]$ 并不是同一个区间。

两个矩恢复点值与区间和

对任意一段差分，用段内从 0 开始的下标 j 维护

$$M_0 = \sum_j d_j, \qquad M_1 = \sum_j j\, d_j.$$

这里的 d_j 表示该段第 j 个元素。节点还维护逻辑长度。

设左段长度为 h，左、右段的矩分别为 L_0, L_1 和 R_0, R_1，拼接后的矩为

$$M_0 = L_0 + R_0, \qquad M_1 = L_1 + R_1 + hR_0.$$

右段下标整体增加 h，所以它对一阶矩额外贡献 hR_0。将整段差分乘以 u 时，两个矩都乘以 u，可以使用乘法懒标记。

对前缀 $d_1, ..., d_t$，局部下标为 $j = i - 1$。差分求和得到

$$a_t = M_0.$$

记 $F(t) = \sum_{i=1}^{t} a_i$，交换求和顺序可得

$$F(t) = \sum_{i=1}^{t} (t - i + 1) d_i = tM_0 - M_1.$$

规定 $F(0) = 0$。操作三的答案是 $F(r) - F(l-1)$，读取某个原序列元素则只需查询差分前缀的 M_0。

操作一的右端修正

直接替换 $d_{l+1}, ..., d_r$ 后，它们的总和可能改变。记

$$\delta = M_0^{\text{new}} - M_0^{\text{old}}.$$

因为 a_l 没有改变，a_r 的变化恰为 δ。若不再处理，所有 $a_{r+1}, a_{r+2}, ...$ 也会增加 δ，违反题目对区间外元素的要求。

若 $r < n$，补上

$$d_{r+1} \leftarrow d_{r+1} - \delta.$$

则差分总变化从位置 $r+1$ 起归零，右侧全部原序列元素恢复原值。若 $r = n$，不需要保存越界的修正。

因此操作一只需要提取种子、构造并替换一段差分，以及至多一次单点加法。修正只依赖目标段新旧的 M_0，不需要 M_1。

操作二的两个边界

先读取本次操作前的 a_{l-1}, a_r，其中 $a_0 = 0$。令 $w = u - 1$。首先执行

$$d_i \leftarrow u d_i \qquad (l \le i \le r),$$

再执行边界加法

$$\begin{aligned} d_l &\leftarrow d_l + w a_{l-1} + v, \\ d_{r+1} &\leftarrow d_{r+1} - w a_r - v \qquad (r < n). \end{aligned}$$

在区间内部，两个相邻原序列元素同时接受同一仿射变换，常数项相消，差分只乘以 u。左边界的目标差分是

$$u a_l + v - a_{l-1},$$

它比 $u(a_l - a_{l-1})$ 多出 $w a_{l-1} + v$。右边界的目标差分是 $a_{r+1} - u a_r - v$，相对旧值的变化正是 $-w a_r - v$。

当 $l = r$ 时两个边界仍是不同的位置，上式直接适用。$u = 0$ 是合法情况；乘法懒标记及其组合必须允许零，不能依赖乘法逆元。所有边界公式中的 a 都取自更新前的版本。

可共享的权重平衡拼接树

所需操作已经转化为：按位置分裂与拼接、提取区间、替换区间、区间乘法、单点加法、查询两个矩，以及重复一个已存在的序列。

使用叶子保存差分元素、内部节点表示左右序列拼接的权重平衡树。节点的权重取它代表的**逻辑元素个数**，维持左右权重的常数比例平衡，树高为 $O(\log n)$。实现标准的平衡 `join` 与 `split`，一次分裂或拼接访问、新建 $O(\log n)$ 个节点。

分裂和拼接只复制受影响的路径，其余子树共享。共享节点不可直接修改：写入、旋转或下传懒标记时，应先得到可独占的新节点。未修改的子树继续共享即可。节点只需维护左右儿子、逻辑长度、两个矩、乘法标记和引用计数等常数信息。

提取种子树 T 后，构造所需长度 $L = r - l$：

1. 若当前逻辑长度小于 L，新建一个左右儿子均为当前根的父节点，使长度翻倍。
2. 继续倍增，直到逻辑长度至少为 L。
3. 按位置分裂，保留前 L 项。

倍增时两个儿子等重，新父节点立即满足平衡条件，每次仅新增一个节点。倍增次数为 $O(\log n)$，停止时逻辑长度小于 $2L$；最后一次截取也只需 $O(\log n)$ 时间和新节点。因此不能把重复过程写成“每复制一份种子做一次拼接”，那会产生与重复次数相关的代价。

将所得树替换到 $[l+1, r]$ 后，再修正 d_{r+1}。仿射操作是区间乘法和至多两次边界加法，查询是两个前缀的矩查询。忽略释放旧节点的工作，每次操作访问或新增 $O(\log n)$ 个节点。

引用计数、总时间与空间

只需保留当前版本。对共享图中的每条父子引用以及每个活动根分别计数；父节点的两个儿子即使指向同一物理节点，也必须计作两条引用。释放根或删除父子边时减去相应引用。引用数降为零的节点才可回收，并继续释放它的儿子；空闲节点应复用。

必须先持有新根需要的引用，再释放旧根。否则新旧根共享的子树可能被提前回收。临时种子根、倍增中间根、截取后不用的部分和被替换区间也都要及时释放。

一次覆盖可能回收一大段旧树，因此**单次操作的最坏耗时可能为线性**。但每个分配的节点最多回收一次：初建 $O(n)$ 个节点，后续每次操作新增 $O(\log n)$ 个节点，所以把回收费用计入总分配量后，总时间为

$$O(n + m \log n).$$

这是摊还界，不能称为每次操作最坏 $O(\log n)$。

空间为 $O(n)$。将当前根的共享图展开为普通二叉树，它恰有 n 个逻辑叶子，最多 $2n-1$ 个节点，因此其中不同物理节点也不会更多。操作过程中只需保留常数个总逻辑长度为 $O(n)$ 的根；倍增目标也小于两倍所需长度。及时释放临时引用并复用空闲槽后，节点池峰值仍为 $O(n)$。

这种持久化只用于实现安全共享。保留全部历史根，或只分配而从不回收，会把空间上界增加到 $O(n + m \log n)$，不再具有满分算法的线性空间保证。

部分分路线

第 1 组：原序列直接模拟

从前两个窗口求出公差 $c = W_{l+1} - W_l$，随后从左到右生成各个目标窗口。每个新窗口恰有一个未知的新加入元素，用目标窗口和减去其余 $k-1$ 项即可确定它。配合滑动和，每次校准为 $O(r - l + 1)$。

也可以使用开头推导的 $a_j = a_{j-k} + c$ 递推。仿射与查询直接遍历，得到总时间 $O(nm)$、空间 $O(n)$ 的算法，适合第 1 组。

第 2 组：仿射线段树

没有操作一时，普通线段树维护区间和与仿射懒标记即可。设已有标记为 $x \mapsto px + q$，再接上 $x \mapsto ux + v$，组合为

$$x \mapsto (up)x + (uq + v).$$

初始化 $O(n)$，每次操作 $O(\log n)$，空间 $O(n)$。

第 3 组：等差赋值与仿射
所有校准均有 $k = 1$。先取旧值

$$A = a_l, \qquad B = a_{l+1} - a_l.$$

校准后，整个 $[l, r]$ 满足

$$a_i = A + B(i - l) = (A - Bl) + Bi.$$

在线段树上统一维护懒变换

$$a_i \leftarrow \mu a_i + q_0 + q_1 i.$$

等差赋值对应 $(\mu, q_0, q_1) = (0, A - Bl, B)$；仿射操作对应 $(u, v, 0)$。已有标记为 (μ, q_0, q_1)，接上新标记 (ν, p_0, p_1) 后，组合为

$$(\nu\mu, \nu q_0 + p_0, \nu q_1 + p_1).$$

对节点 $[x, y]$，用长度和 $\sum_{i=x}^{y} i = (x + y)(y - x + 1)/2$，即可在 $O(1)$ 时间更新区间和。总时间 $O(n + m \log n)$，空间 $O(n)$。

提供的 `partial_special.cpp` 将上述路线组合：一般 $k > 1$ 时取出原数组区间、线性递推后写回，其余操作使用线段树。它在算法上覆盖前 3 组共 50 分；最终 4 秒版实测 70 分。

第 4 组：一般操作的中等规模
共享树若省去回收，只分配路径和倍增节点，仍有 $O(n + m \log n)$ 时间、$O(n + m \log n)$ 空间，可作为中等规模的一条路线。低常数模拟也可能通过部分一般数据，但该组的完整得分必须由实际测试确认。本组不限定必须采用某一种结构。

第 5 组提高一般操作的规模，要求算法同时满足大规模时间和空间限制。各部分分程序以及关闭回收后的程序得分，均应以本版验收记录为准。

直接维护原序列的独立满分参考

还可以在可共享拼接树中直接保存 a，只维护区间和及普通仿射标记。

取旧序列中的长为 k 的块

$$S = (a_l, a_{l+1}, ..., a_{l+k-1}),$$

并令 $c = a_{l+k} - a_l$。窗口条件给出 $a_{i+k} = a_i + c$，故校准后区间的第 q 个块为 $S + qc$。这里对块加常数表示块中每一项均加上该常数。

若当前段含从第 0 块起的连续 s 个块，那么后面的 s 个块等于当前段整体加上 sc。所以可以倍增：把当前段与“当前段整体加 sc 的共享副本”拼接，再令块数变为 $2s$。最后截取前 $r - l + 1$ 项，替换原数组的整个 $[l, r]$。

虽然种子只有 k 项，生成的第 $k+1$ 项仍为 $a_l + c = a_{l+k}$，恰好保留题目指定的全部 $k+1$ 项。

`std_avl.cpp` 按这一思路直接保存原数组，以 AVL 高度保持平衡，使用独立的引用计数和仿射懒标记。它不需要差分、矩或边界修正，可用于与标准程序交叉核对。倍增仍新增 $O(\log n)$ 个节点；配合可共享 AVL 的分裂、拼接及回收，同样有 $O(n + m \log n)$ 摊还总时间和 $O(n)$ 空间。

实现检查要点

- 差分种子为 $[l+1, l+k]$，替换范围为 $[l+1, r]$，长度为 $r-l$。
- $k = r - l$ 可直接返回；最小合法校准区间长度为 2。
- 操作一右侧只修正 d_{r+1}，增量是目标段总和变化的相反数。
- 操作二的边界原值必须在更新前读出，$r = n$ 时忽略右侧修正。
- M_1 使用局部 0 基下标；拼接右段要加上左段长度对应的平移量。
- 平衡依据逻辑长度，不能使用共享图中的不同物理节点数代替。
- 一个节点的两个儿子可以相同，但引用计数和懒标记下传仍要按两条边处理。
- 模减法、$u - 1$ 和负修正均需规范化；零乘法标记不能当作“没有标记”。
- 先持有新引用再释放旧引用；及时回收并复用临时节点，避免节点池随操作总数增长。
- 两个模剩余类相乘以及长度乘模剩余类应使用至少 64 位中间类型；多个乘积相加时及时取模。

上下左右 题解

记 D 为正右移次数，R 为右上、右下的总次数，L 为左上、正左、左下的总次数，V 为正上、正下的总次数。步数与横向位移给出

$$D + R + L + V = N^2 - 1, \qquad D + R - L = N - 1,$$

故 $D = (N^2 + N - 2 - C)/2$，其中 $C = V + 2R$ 为偶数。最优值为

$$M(N) = \begin{cases} (N^2-1)/2, & N \text{ 为奇数;} \\ (N^2+N-2)/2, & N \equiv 2 \pmod 4; \\ (N^2+N-4)/2, & N = 4, 8; \\ (N^2+N-6)/2, & N \equiv 0 \pmod 4,\ N \geq 12. \end{cases}$$

五档做法

实现	适用范围	累计保证分数	复杂度
`partial_dfs_10.cpp`	$N \le 3$	10	枚举简单路径，指数级
`partial_subset_25.cpp`	$N \le 4$	25	$O(2^{N^2}N^2)$ 时间、空间（八邻接度数为常数）
`partial_odd_45.cpp`	上一档或奇数 N	45	小规模子集 DP，其余 $O(N^2)$
`partial_bands_65.cpp`	上一档或 $N \equiv 2 \pmod 4$	65	小规模子集 DP，其余 $O(N^2)$
`std_maximum.cpp`	全部	100	$O(N^2)$ 时间、空间

前两档直接维护“已访问格子集合、最后一个格子”，转移时累加是否走了 `D`。最后一个格子必须是右下角，可沿最优转移恢复路径。DFS 只用于 $N \le 3$。子集 DP 是独立求最优值的算法，不依赖上面的公式。

奇数与两列条带

按列号奇偶染色。奇数 N 时两种颜色的格子数相差 N，起终点都属于较多的颜色。只有竖直步保持颜色，因此 $V \ge N-1$，得到 $D \le (N^2-1)/2$。逐行蛇形遍历恰好从左上到右下，达到等号。

当 $N \equiv 2 \pmod 4$ 时，将列分成相邻的两列条带。每行先访问左格再访问右格，相邻行用左上或左下连接；条带之间用 `D` 连接，并交替向下、向上。条带数 $N/2$ 为奇数，终点为右下角。整个构造没有竖直步和右斜步，故 $C=0$，已达到统一上界。$N=2$ 也由该构造覆盖。

四的倍数

若 $C=0$，行内的 `D` 被强制组成上述两列条带蛇形。条带数为偶数时终点不在右下角，因此 $C \ge 2$。$N=4,8$ 有 $C=2$ 的固定构造，代码直接列出两条串。

对于 $N \ge 12$，还需要证明 $C=2$ 不可能。分别排除 $(V,R)=(0,1)$ 和 $(2,0)$：前者由两列条带内的匹配与条带端点位置排除；后者由缺失 `D` 所在行的相邻性、顶底边界状态交替，推出活动条带数至少 $N-4$，但最多 $N/2$，因而只能 $N \le 8$。完整推导见 README 的证明。因此 $C \ge 4$。

`fourVertical` 给出恰好四个竖直步、零个右斜步的构造。使用零基行列号，令 $m=N/2$，第 j 个两列条带定义

$$a_j = \min(j+1, m-2), \quad b_j = \max(N-2-j, m+1),$$

$$d_j = \begin{cases} -1, j \text{ 为偶数且 } j < m-3; \\ 1, \text{ 其他 }. \end{cases}$$

依次访问五段: 顶部条带、从右到左的上通道、中部条带、从右到左的下通道、底部条带。条带的每个完整行先左格再右格，向上或向下交替扫描。上通道占用每条带 a_j 行及必要的邻行，下通道占用 b_j 行及必要的邻行；条带段排除这些格子。各段行区间和方向在代码 `stripe` 的实参中明确给出，互不重叠且覆盖全部 N^2 格。五段的四次连接为竖直步，其余只有 `D` 和向左的三个方向，故 $C = 4$ 并达到上界。

路径输出本身长 $N^2 - 1$，所以 $O(N^2)$ 时间最优。程序保存路径坐标和答案串，空间为 $O(N^2)$。

最优值证明

以下保留来源产物的完整上界证明与构造说明，路径名已适配当前目录。正确性依赖此证明；有限测试用于检查实现。

给定 N⩾2，起点 (1,1)，终点 (N,N)，允许八邻接，恰好遍历全部格子一次。记真正可能的最大正右移次数为 M(N)。

精确结论为

$$M(N) = \begin{cases} (N^2-1)/2, & N \text{ 为奇数}, \\ (N^2+N-2)/2, & N \equiv 2 \pmod 4, \\ (N^2+N-4)/2, & N = 4 \text{ 或 } 8, \\ (N^2+N-6)/2, & N \equiv 0 \pmod 4,\ N \geq 12. \end{cases}$$

以下给出上界证明。达到各上界的构造实现于 `sol/std_maximum.cpp`。该程序的输入是 T，随后每组只有 N；输出最大 K 及一条达到它的路径。构造时间、空间均为 O(N^2)。

统一计数。

设 D 为正右移次数，R 为右上、右下的总次数，L 为左上、正左、左下的总次数，V 为正上、正下的总次数。总步数和横向位移分别给出

$$D + R + L + V = N^2 - 1, \qquad D + R - L = N - 1.$$

所以

$$D = \frac{N^2 + N - 2 - C}{2}, \qquad C := V + 2R.$$

C 必为偶数。求最大 D 等价于求最小 C。

奇数 N。

按列号奇偶染色。两种颜色的格子数相差 N，起终点均属于数量较多的颜色。只有竖直移动不改变颜色，因此至少需要 N−1 次竖直移动:

$$V \geq N-1.$$

故 D⩽(N^2−1)/2。逐行左右蛇形遍历达到等号。

偶数 N，C=0 的结构。

此时只允许 D、A、Q、Z。把列分成 (1,2),(3,4),⋯ 的两列条带。

第一列的每个格子都必须向右走，所以第一、二列之间所有同行的 D 都被使用。第二列所有格子的入边已经确定，第三列不能再向左进入第二列。于是前两列必须作为一个连续段遍历。

由于每一行都走 D，行间只能用 Q 或 Z 相连；覆盖所有行就必须从顶到底或从底到顶。逐条带归纳，整条路径被强制为两列条带蛇形。

条带数 N/2 为奇数时，终点恰好在右下角。因此 N≡2(mod4) 时 C=0 可达，最大值是 (N^2+N−2)/2；N≡0(mod4) 时 C=0 不可达。

为什么 C=2 只能在 N=4,8 时出现?

下面对 N⩾12 的情形作反证；N=4,8 已有 C=2 的构造。

先排除 V=0,R=1。

无竖直边时，每个奇数列切口有 N 条向右边，每个偶数列切口恰有一条向右边、没有向左边。每个两列条带中的 N 条向右边必须构成两列格子的完美匹配。如果其中恰有一条右斜边、其余均为同行 D，就不可能是完美匹配。因此那条右斜边只能位于条带之间。

但条带内全是 D，必须按行单调遍历整个高度，出入口都在顶行或底行。条带间的一条右斜边会把下一个条带的入口移到内部行，无法遍历其全部行。故 V=0,R=1 不可能。

现在只需研究 V=2,R=0。

下文用零基行号 0,⋯,N−1，列号仍用一基。设第 c 列有 v_c 条竖直边，第 c、c+1 列之间共有 s_c 条路径边。由每个格子的度数求和，

$$s_1 = 2N-1-2v_1, \qquad s_c = 2N-2v_c-s_{c-1} \quad (1<c<N).$$

每个切口向右次数比向左次数多 1，所以 s_c⩾1。

由递推可得，两条竖直边必须分别位于一个奇数列 a 和它后面的一个偶数列 b。把从列 a 到列 b 的 p=(b−a+1)/2 个两列条带称为活动条带。

活动区中： - 奇数列切口有 N−1 条 D、N−2 条向左边； - 两个活动条带之间有两条 D、一条向左边； - 活动区外的两列条带之间只有一条 D。

因此，每个活动条带恰有一行没有内部的 D，记为 h_j。相邻活动条带间唯一的向左边，只能从右侧条带缺口行的左格，走到左侧条带缺口行的右格，所以

$$|h_{j+1}-h_j| \leq 1.$$

关键是证明

$$\min_j h_j \le 2, \qquad \max_j h_j \ge N-3.$$

下面严格证明第一条。假设所有 h_j≥3。于是所有两列条带的第 0、1、2 行都使用内部 D，而且第 0、1 行没有竖直边。

在每个两列条带之间的切口，用两个比特 (x,y) 表示第 0、1 行是否有跨向下一条带的 D。左边界用虚拟状态 (1,0) 表示起点；右边界状态为 (0,0)。

考察一个条带，设左右状态分别为 (a,b)、(c,d)。

- 若 a=c=0，顶行只能与第二行相互连接，与这两行的 D 组成独立四格环，不合法。
- 若 a=1,c=0，顶行向下连接第二行，故 b=0，只能有 10→00 或 10→01。
- 若 a=0,c=1，同理只能有 00→10 或 01→10。
- 若 a=c=1，而 b=d=0，第二、三行被迫组成独立四格环，不合法。
- 11→11 也不合法：两侧切口的两条 D 都已经用在顶两行，因此该条带必在活动区内部，没有竖直边；下面所有行只能组成一条从缺口右格到缺口左格的内部路径。将其他行的 D 收缩为一个点，得到一条行号链，缺口行拆成两个端点。两端点之间的简单路径只能直接走 A，或经过一个相邻完整行再折返，最多覆盖两个行号。下面却有 N−2≥4 行，矛盾。

因此所有合法转移都在如下两个集合间交替：

$$\{10\} \quad \leftrightarrow \quad \{00, 01, 11\}.$$

从左边界 10 到右边界 00，需要奇数个条带。但 N≡0(mod4)，条带数 N/2 为偶数，矛盾。因此 min h_j≤2。

对底部两行作上下镜像分析，虚拟边界状态从 00 到 10，仍要求奇数个条带。同理得到 max h_j≥N−3，证明了 (2)。

结合 (1)、(2)，

$$p - 1 \ge \max h_j - \min h_j \ge N - 5,$$

于是

$$p \ge N - 4.$$

另一方面 p≤N/2，因此 N≤8。

所以，N≡0(mod4)、N≥12 时，C=0 与 C=2 均不可能，必有 C≥4。这给出

$$D \le \frac{N^2 + N - 6}{2}.$$

上界均可达到。

- 奇数 N：逐行蛇形，C=N−1。
- N≡2(mod4)：两列条带蛇形，C=0。

- N=4：路径 DDDZDXAQAXDZDDD，恰有 8 次 D。
- N=8：`sol/std_maximum.cpp` 中给出一条 34 次 D 的固定路径。
- $N\equiv 0 \pmod 4$、$N\geqslant 12$：代码的 fourVertical 构造恰有四次竖直移动，没有向右斜移，因此 C=4，达到上界。

一般的四竖步构造把每个两列条带分成顶部、中部、底部，以及两条向左返回的通道。设 m=N/2，第 j 个条带使用

$$a_j = \min(j+1, m-2), \quad b_j = \max(N-2-j, m+1),$$

$$d_j = \begin{cases} -1, & j \text{ 为偶数且 } j < m-3, \\ 1, & \text{其他}. \end{cases}$$

遍历顺序为顶部条带、上通道、中部条带、下通道、底部条带。各区域不交且覆盖全部格子，条带内每行先左格后右格；只有五个大段之间的四次连接是竖步，其他步都为 D 或向左的三个方向。代码直接实现这些区间与通道，完整给出了达到上界的构造。

存储空间 题解

完整算法的时间复杂度为 $O(n + q\log^2 n)$，空间复杂度为 $O(n)$。

本题的两个难点是：任意分割树的一个区间可能包含线性多个规范节点；这些节点的需求次数变化，又会批量改变最优费用与方案数。下面依次把它们转化成两条边界路径、重链上的矩阵，以及由最小值控制的四种矩阵乘积。

1. 将目标区间转化成节点需求

树节点区间具有层级性质：两个区间相交时，其中一个必然包含另一个。

对目标区间 $J = [l, r]$，令 $D(J)$ 为所有完整包含于 J 的极大树节点。它们两两无交，并集为 J，称为 J 的规范分解。

引理：一个节点集合 P 能覆盖 J，当且仅当它能分别覆盖 $D(J)$ 中的每个节点。

充分性直接将各块的覆盖合并。必要性如下：一个用于覆盖 J 的节点必然完整包含于 J；沿祖先向上寻找仍包含于 J 的最高节点，它就落在唯一一个规范块中。因此，把覆盖 J 时选出的节点按所在规范块分组，每组正好覆盖对应的块。

令

$$c_v = \#\{J \in T : v \in D(J)\}.$$

只要保证所有 $c_v > 0$ 的节点都能被完整覆盖，就等价于满足原问题。这里必须维护引用次数；多个不同原区间可能共享同一个规范块，删除其中一个不能直接清除该块的需求。

任意树上的规范分解可能有 $\Theta(n)$ 个节点。例如反复将 $[i, n]$ 分成 $[i, i]$ 与 $[i+1, n]$，则 $[1, n-1]$ 的规范分解包含 $n-1$ 个叶子。因此不能逐个枚举全部规范块。

2. 用最高包含祖先确定两条边界路径

对 $J=[l,r]$，先找到：

- a：第 l 个叶子的最高祖先，满足 $I_a\subseteq J$；
- b：第 r 个叶子的最高祖先，满足 $I_b\subseteq J$。

若 $a=b$，规范分解只有这个节点，只修改一次 c_a。

否则令 $z=\mathrm{LCA}(a,b)$。规范分解恰好由以下节点组成：

1. a,b；
2. 从 a 向 z 上行时，若经过的孩子是左孩子，就加入其右兄弟；
3. 从 b 向 z 上行时，若经过的孩子是右孩子，就加入其左兄弟。

后两项都**不包含父亲恰为 z 的那条边**。

证明可以分别看 z 的左右子树。左侧从 a 开始，沿途加入位于路径右侧的完整子树，恰好补齐从 l 到 z 的左子树最右端；右侧对称。由于 a,b 已经是最高完整包含祖先，路径上更高节点都不完整包含于 J，所以这些兄弟子树不仅在 J 内，而且确实是极大的规范块。两边互不交叠。

不能直接从端点叶子开始使用这个规则。若整个根区间就是目标，它应当只产生根节点需求；拆成许多更小节点会改变题意，排除直接存储根的合法方案。

2.1 在线性空间内寻找最高包含祖先

按子树叶子数选择重孩子，数量相等时选择右孩子。任意根到叶路径至多经过 $O(\log n)$ 条轻边。

从端点叶子向上考察重链。如果当前链头完整包含于 J，就可以越过整条链，再判断链头的父亲；否则，在当前链头到当前位置之间二分，找出第一个完整包含于 J 的节点。沿祖先方向，“完整包含”的判定一旦失败就不会再次成功，因此二分具有单调性。

最多进行 $O(\log n)$ 次轻边跳转，并且只在最后一条链上二分一次，总时间为 $O(\log n)$。LCA 也用重链跳转在 $O(\log n)$ 时间内求出。二者不需要 $O(n\log n)$ 空间的倍增祖先表。

3. 局部最优状态与无重计数

把一个最优状态表示为二元组 (d,k)，分别为最小费用与达到该费用的方案数。定义两种运算：

- $x\otimes y$：费用相加，方案数相乘；
- $x\oplus y$：保留费用较小者；若费用相同，保留此费用并将方案数相加。

方案数始终对 998244353 取模。记 $\mathbf{1}=(0,1)$、$\mathbf{0}=(+\infty,0)$，并把费用 w_v 简写为二元组 $(w_v,1)$。

对节点 v 定义：

- X_v：满足所有严格子孙中的规范需求，暂时忽略 c_v；
- B_v：满足上述需求，并额外完整覆盖 I_v；

· A_v：满足包含自身在内的实际需求，即 $c_v = 0$ 时取 X_v，否则取 B_v。

对内部节点，记左右儿子为 L, R：

$$X_v = A_L \otimes A_R,$$

$$B_v = (w_v \otimes X_v) \oplus (B_L \otimes B_R).$$

第一个分支选中 v，但仍需满足两棵子树各自的需求；第二个分支不选 v，改为完整覆盖两个儿子。

选择 v 不能用于精确覆盖它的任何严格子孙，因为范围过大。费用又严格为正，所以仅满足严格子孙需求时，最优方案一定不选 v。这证明了 X_v 的转移，也说明正费用条件不可随意删去。

两个 B 分支分别包含和不包含 v，对应的节点集合互不相交。因此费用相同时直接加方案数。固定分支后，节点集合唯一分解成两个子树中的集合，故两边方案数相乘。整个递推统计的是存储节点集合，不统计它对目标的不同拼接表示。

叶子处为

$$X_v = \mathbf{1}, \qquad B_v = (w_v, 1).$$

最终答案为根的 A。按树高归纳即可证明状态正确，再结合规范分解引理得到原问题的答案。

关键不变量是：$\boldsymbol{X_v, B_v}$ **都不依赖节点自身的** $\boldsymbol{c_v}$。自身是否被需求，留给父亲在使用 A_v 时判断。根的 c_1 单独控制最终选用哪个状态。

4. 把一条重链写成矩阵乘积

设内部节点 v 的重孩子为 h，轻孩子为 s，并令

$$C = \begin{cases} X_s, c_s = 0, \\ B_s, c_s > 0. \end{cases}$$

将轻子树视为已知系数，则

$$\begin{pmatrix} X_v \\ B_v \end{pmatrix} = M_v \begin{pmatrix} X_h \\ B_h \end{pmatrix}.$$

矩阵使用上一节定义的二元组加乘法。当 $c_h = 0$ 时，

$$M_v = \begin{pmatrix} C & \mathbf{0} \\ w_v \otimes C & B_s \end{pmatrix};$$

当 $c_h > 0$ 时，

$$M_v = \begin{pmatrix} \mathbf{0} & C \\ \mathbf{0} & (w_v \otimes C) \oplus B_s \end{pmatrix}.$$

每条重链最终到达一个叶子 t。将链上内部节点的矩阵从上到下相乘，再作用于

$$\begin{pmatrix} \mathbf{1} \\ (w_t, 1) \end{pmatrix},$$

就得到链头的 (X, B)。也可以让链尾叶子在线段树中存单位矩阵，再使用同一个末端向量。

矩阵乘法不可交换，所有区间合并和查询都必须保留从链头到链尾的顺序。

链头向父链只报告 (X, B)；链头自身的需求次数在其父亲处使用。这使得修改大量轻孩子根的 c 时，不必进入这些轻子树重新计算。

5. 需求次数放在哪里，边界路径如何修改

逻辑上，每个节点有一个计数 c_v。实现上可按它与父亲的关系存储：

- 根的计数单独保存；
- 若 v 是重孩子，其计数保存在父亲的重孩子计数中，修改时重建父亲一个矩阵；
- 若 v 是轻孩子，其计数保存在父亲所在链的位置上，供后续的链区间操作维护。

考虑第 2 节的兄弟节点更新：

- 边界路径经过重边时，兄弟节点是轻孩子，修改的是父亲链位置上的轻孩子计数；
- 经过轻边时，兄弟节点是重孩子，只需做一个点修改。

轻边只有 $O(\log n)$ 条，因此后一类只有 $O(\log n)$ 个点。前一类在每条重链上形成连续段，但需要按方向过滤：左边界只取右兄弟，右边界只取左兄弟。

将链位置分为“轻孩子在左侧”和“轻孩子在右侧”两类，一次原区间操作就转化为 $O(\log n)$ 个带方向的链区间加减、少量点修改，以及端点 a, b 的计数修改。

同一个规范节点在一次分解中只出现一次。合法删除保证每个被减少的计数原本为正，因此全部计数在更新中保持非负。

6. 最小值与四种假想乘积

困难在于：一个链区间的轻孩子计数统一加减，可能使许多位置在 $c_s = 0$ 与 $c_s > 0$ 之间切换，改变很多矩阵。

先看一种方向。对某个维护区间，记这一方向计数的最小值为 μ，保存两种假想矩阵乘积：

- 全部位置都使用“轻孩子计数为正”的矩阵；
- 恰好在计数等于 μ 的位置使用“计数为零”的矩阵，其余位置仍使用正计数矩阵。

由于所有计数非负，若 $\mu > 0$，真实乘积是第一种；若 $\mu = 0$，真实乘积是第二种。

当整段这一方向的计数统一加上 Δ 时，取得最小值的位置不变，所以两种假想乘积都不变。只需修改 μ 与懒标记；真实乘积可以由新最小值重新选择。

实际有左右两类位置，每类都有上述两种模式，因此一个线段树节点保存四种乘积

$$P_{00}, P_{01}, P_{10}, P_{11}.$$

某一位为 0 表示该方向全用正计数矩阵，为 1 表示该方向的最小值位置用零计数矩阵。两类位置互不重叠；同一个位置只属于其中一类。

6.1 合并公式

设两个相邻维护区间为 U, V。对方向 d，父区间最小值为

$$\mu_d = \min(\mu_d^U, \mu_d^V).$$

要构造父区间指定的模式：

- 若某子区间的最小值等于 μ_d，该子区间继承父区间在方向 d 的模式位；
- 若某子区间的最小值大于 μ_d，它在此方向不含父区间的最小值位置，模式位强制为 0。

分别得到两个子区间的模式后，按原顺序将它们的矩阵乘积相乘。对四种父模式分别处理，即可在常数时间合并。

区间真实模式的第 d 位当且仅当 $\mu_d = 0$ 时为 1。某一方向没有任何位置时，应单独表示“该类不存在”，对它不执行数值加减；不可让普通数值哨兵在懒标记中反复变化。

若某个轻子链的 (X, B) 改变，或者某点的重孩子计数发生零与正的切换，就下放沿途懒标记，重建这个位置的零计数矩阵、正计数矩阵及四种模式，再向上合并。

这样，方向区间加减、矩阵系数点修改和链乘积查询，都可以在 $O(\log n)$ 时间内完成。

7. 批量完成一次操作，避免多一层对数

若每做一个链区间更新，就立刻沿所有父链传播变化，会得到 $O(\log^3 n)$ 的做法。

应该先执行本次操作的全部直接改动，再统一传播。所有直接改变的矩阵位置，都在从 a, b 的父亲到根的两条路径上。规范兄弟节点自身的 (X, B) 不变，只有它在路径上父亲的系数变化，所以不会额外进入大量旁支。

收集这两条路径经过的重链，去重并按链头深度从大到小处理。对每条链：

1. 它自己的直接更新已经完成；
2. 更深的相关轻子链已经报告了新状态，并更新了本链相应位置的系数；
3. 查询本链的真实矩阵乘积，得到链头的新 (X, B)；
4. 若状态变化，向父链报告一次；若未变化，可以省去这次点重建。

这样每条链只求值、报告一次。比较状态是否变化时，必须同时比较费用与模后的方案数。

两条根路径的并只涉及 $O(\log n)$ 条重链。每条链最多接受两条边界路径带来的常数个连续段、常数个轻边或端点点修改，以及常数个相关轻子链报告。因此每条链处理 $O(\log n)$ 时间，总计

$$O(\log^2 n)$$

每次操作。按深度排序 $O(\log n)$ 条链所需的时间也包含在此界内。

如果规范分解只有根，只有根的计数变化，链头 (X, B) 本身不受影响，直接重新选择最终答案即可。

8. 总体复杂度与实现边界

预处理叶序、子树区间、叶子数、重链与初始 DP，均为 $O(n)$。维护结构是线性的矩阵线段树，不需要倍增表或为每个原区间保存规范分解。

总体为

$$\boxed{O(n + q \log^2 n) \text{ 时间}, \qquad O(n) \text{ 空间 .}}$$

对 $n = 1$，将对数记成 $\log(n + 1)$ 即可统一复杂度表述。

实现时尤其注意：

- **树遍历不能依赖编号顺序**。根以外的节点编号任意，应建立实际遍历序，再逆序计算子树信息。
- **输入树深度可能为 $\Theta(n)$**。遍历原树、预处理和链构造宜使用显式栈；维护矩阵的线段树递归深度只有 $O(\log n)$。
- **初始 B_v 不是一律 $(w_v, 1)$**。即使没有需求，完整覆盖一个内部节点也可能通过子孙更便宜，或者存在费用相同的多种方案；必须自底向上计算。
- **费用使用 64 位整数**。选择所有叶子是可行方案，故最优费用不超过 $n \cdot 10^9 \le 10^{14}$。无穷大值应留足余量，且不可行项参与乘法时先判断，不直接做无穷大相加。
- **方案数为零不代表不可行**。它可能只是取模后的结果；可行性由费用是否为无穷大判断。
- **c 不是下传给子树的覆盖标记**。它只表示该完整节点的需求。只有最小值维护结构中的区间加减需要下放自身懒标记。
- **轻孩子计数修改不重新计算轻子树**。轻子树只报告与自身计数无关的 (X, B)；否则批量修改会退化。
- **排除 LCA 边、避免重复端点**。$a = b$ 时只修改一次；两条边界路径都排除父亲为 z 的边。
- **矩阵方向必须一致**。链产品按从上到下排列，区间查询左右结果不得交换。
- **常数空间也需检查**。每个维护节点保存四个 2×2 矩阵，可使用紧凑线段树布局，并将费用、方案数分开存放以减少结构体对齐开销。时间与内存限制仍应以目标环境压测为准。

9. 子任务与可行的部分分算法

子任务 1：$n \le 7, q \le 100$，5 分

树节点至多 13 个，枚举全部 2^{2n-1} 个存储集合 P。每个候选的费用为所选节点费用之和，不按节点数量比较。

可以独立检查覆盖：把选中区间 $[a, b]$ 视为一条从 a 到 $b + 1$ 的有向边，则 P 能覆盖 $[l, r]$ 当且仅当 l 能到达 $r + 1$。路径中的区间首尾相接，恰好对应无交覆盖，反之也成立。

预处理每个 P 能覆盖的全部原区间，用位集保存。每次操作后筛选能覆盖所有活动目标的候选，比较费用并计数。此暴力不依赖规范分解和动态树 DP，可用于独立对拍。

子任务 2：$nq \le 2 \times 10^6$，10 分
每次操作沿实际树递归，枚举全部规范节点并修改 c，然后全树重算第 3 节的 DP。每次至多访问 $O(n)$ 个节点，总时间 $O(nq)$，空间 $O(n)$。

子任务 3：精确中点分割，10 分
树高为 $O(\log n)$。一次普通区间递归只访问 $O(\log n)$ 个节点；完整包含时修改当前节点的 c 并停止下降，回溯时合并 DP。

总时间 $O(n + q \log n)$，空间 $O(n)$，不需要重链矩阵和最小值分类。

子任务 4：叶子深度不超过 30，5 分
令实际树高为 $H \le 30$。沿真实区间边界递归，每层只有常数个访问节点，完整包含时停止。与上一档相同的局部 DP 可在 $O(n + qH)$ 时间、$O(n)$ 空间内完成。

子任务 5：所有目标都是树节点，10 分
每个目标只有一个规范块，因此只进行计数点修改。当前原区间集合无重复，同一个节点的直接需求可以用布尔值维护。

用普通动态树 DP：重链上维护单个矩阵乘积，点修改后逐条父链传播。每次至多经过 $O(\log n)$ 条链，每条链处理 $O(\log n)$，共 $O(\log^2 n)$。不需要按方向区间加减，也不需要四种假想乘积。

节点可通过预建的区间到编号映射定位，也可利用最高包含祖先找到；总空间为 $O(n)$。

子任务 6：每个内部节点的左孩子为叶子，10 分
这是一棵向右延伸的梳状树，内部节点依次对应 $[1, n], [2, n], ..., [n-1, n]$。

- 若 $r < n$，区间 $[l, r]$ 的规范分解就是叶子 $l, l+1, ..., r$，对应同一种方向的连续计数加减；
- 若 $r = n$，规范分解只有后缀节点 $[l, n]$，对应一次点修改。

全部内部节点位于同一条重链。只需维护一种方向的最小值，以及“全正”“最小值位置为零”两种矩阵乘积，不需要四种模式，也不需要多条父链之间的传播。

每次操作 $O(\log n)$，总时间 $O(n + q \log n)$，空间 $O(n)$。$n = 1$ 时直接维护唯一节点是否被需求。

子任务 7：每个内部节点的右子树不少于左子树，10 分
平局时也选择右孩子为重孩子，则所有轻孩子都位于左侧。只需一种方向的最小值与两种假想乘积。

仍可能有多条相关重链，需要按第 7 节批量传播；但不需要左右方向同时存在时的四模式合并。总时间 $O(n + q \log^2 n)$，空间 $O(n)$。

子任务 8：只有插入，10 分
此时一个规范节点的需求只会从无变有，可以只处理首次激活。难点仍是跳过已经激活的大量规范块，不能每次显式枚举整个分解。

先按两条边界路径把更新拆成 $O(\log n)$ 个链区间及点。在每种轻孩子方向上，用有序集合保存尚未激活的位置。处理一个链区间时，只枚举落在该区间内的未激活位置，激活后立即从集合删除；重孩子点和端点则直接判断布尔标记。

每个节点最多被首次激活一次，合计 $O(n)$ 次有效激活。每次有效激活使用子任务 5 的普通点修改动态树 DP，花费 $O(\log^2 n)$；每个链区间在有序集合中定位一次，花费 $O(\log n)$。

总体可做到 $O((n+q)\log^2 n)$ 时间、$O(n)$ 空间，无需支持删除时的正计数转零，也无需完整的四模式最小值结构。

子任务 9：目标区间长度不超过 2，5 分

任何规范需求只会落在叶子或拥有两个叶子孩子的节点上。把每个这样的内部节点与它的两个叶子组成一个独立的双叶组件；其余叶子单独成组件。

长度大于 2 的存储节点不能用于精确覆盖任何当前目标，因此最优方案不会选择它们。各组件完全独立。

设一个双叶组件的父费用为 p，左右叶子费用为 u, v；父节点是否被规范需求记为 c，左右叶子是否被规范需求记为 a, b。这些布尔值由对应的引用次数是否大于零得到。

若 $c = 0$，唯一最优方案是选择所有被需求的叶子，贡献费用

$$au + bv,$$

方案数为 1。

若 $c = 1$，比较两个互斥候选：

$$\underbrace{p + au + bv}_{\text{选父节点以及所有必选叶子}}, \qquad \underbrace{u + v}_{\text{不选父节点，选择两个叶子}}.$$

两者较小者为组件最优费用；相等时有两种方案，否则只有一种。若两个叶子都被需求，由于 $p > 0$，第二个候选严格更优。

孤立叶子的贡献为：有需求时选择它，费用为自身权值；没有需求时不选。两种情况的方案数均为 1。

全局最优费用是组件费用之和；若 z 个双叶组件出现平局，全局最优方案数为 2^z。预处理 2 的幂，即可常数时间读取方案数。

操作时，单点修改对应叶子的引用次数。长度为 2 的区间若恰好对应双叶组件的父节点，修改父节点引用次数；否则它跨过组件边界，规范分解为两个叶子，应分别修改这两个叶子的引用次数。后者尤其不能只用布尔值：相邻的两个长度为 2 的原区间可能共享同一个叶子，删除其中一个后，该叶子仍可能被另一个区间需求。

预先按真实树形建立叶子到组件的映射，每次操作只影响常数个组件。更新前扣除旧费用、旧平局贡献，修改次数后加入新贡献。总时间 $O(n+q)$，空间 $O(n)$，与原树高度无关。

子任务 10：无额外约束，25 分

使用完整算法：最高包含祖先与边界路径、忽略自身需求的 (X, B)、带方向的计数更新、四种最小值分类矩阵乘积，以及每次操作的统一链传播。

总时间 $O(n + q \log^2 n)$，空间 $O(n)$。

10. 正确性验证建议

小规模可直接枚举全部存储集合，并用区间可达性检验覆盖，与全树 DP、完整动态程序交叉比较。应特别覆盖：空需求、只需求根、费用相等的直接存储与向下拼接、祖孙需求同时存在、不同原区间共享规范块，以及反复删除后仍保留其他引用的情况。

结构方面需要包含线性深度、左右重孩子方向交替、非拓扑顺序编号、两端点处于同一重链或不同重链，以及 $a = b$ 的完整节点需求。四模式维护还应核对整段加减、只更新某一方向、方向不存在和子链状态变化后的点重建。

这些检查针对算法正确性；实际运行时间和内存应在完整程序及目标评测环境上另行测量。